

VortiQ Studio

Technical Tools Mastery & Comprehensive Interview Prep Guide

Document Purpose: This comprehensive reference guide covers every technology, tool, architecture design pattern, and technical interview question associated with the **VortiQ Studio** full-stack project (Java 21/24 Spring Boot 3.3 REST API + React 18 Vite Frontend). Use this guide to master interviews and explain your project with 100% technical authority.

PART 1: Understanding All Tools & Technologies Used

Technology / Tool	Role in Project	Key Concept to Explain in Interviews
Java 21 / 24	Core Backend Programming Language	Modern JDK features, Strong Typing, OOP Principles, Exception Handling
Spring Boot 3.3	Enterprise Backend Framework	Inversion of Control (IoC), Dependency Injection (DI), Auto-configuration
Spring Data JPA	Data Access & Repository Layer	Object-Relational Mapping (ORM), JpaRepository abstractions, Dynamic SQL
Hibernate ORM	JPA Implementation Provider	Entity lifecycle management, DDL auto-generation, <code>@Entity</code> , <code>@Id</code> , <code>@</code>
H2 Database	In-Memory Relational DB Engine	Zero-config persistence, zero external DB dependencies, <code>jdbc:h2:mem:</code>
Apache Maven 3.9+	Java Dependency & Build Tool	<code>pom.xml</code> , dependency management lifecycle (<code>clean</code> , <code>compile</code> , <code>test</code> , <code>package</code>)
React 18	Frontend User Interface Framework	Component-driven architecture, Hooks (<code>useState</code> , <code>useEffect</code> , <code>useCall</code>)
Vite 5	Frontend Build & Bundling Tool	Fast HMR (Hot Module Replacement), ES Modules, production bundling
Lucide React	Icon Library	Scalable SVG vector icon integration for visual UI elements.
Vanilla CSS & Glassmorphic Design	Styling & UI Tokens	CSS Custom Properties (Variables), <code>backdrop-filter: blur()</code> , Grid, Flexbox
Docker & Docker Compose	Containerization & Cloud Hosting	Multi-Stage Dockerfile (Node 20 -> Maven 3.9 -> JRE Alpine), lightweight
Git & GitHub	Version Control & Collaboration	Repository tracking, <code>.gitignore</code> rules, human commit hygiene, remote or

PART 2: Comprehensive Technical Interview Questions & Answers

Q1: How does Spring Boot Inversion of Control (IoC) and Dependency Injection (DI) work in this project?

Answer: In Inversion of Control (IoC), the Spring Container controls the creation and lifecycle of Java objects (Beans) rather than the application creating them with `new`. Dependency Injection (DI) is the pattern used to supply dependencies.

In our project, **TaskController** requires **TaskService**, and **TaskService** requires **TaskRepository**. Spring automatically injects these dependencies via Constructor Injection:

```
public TaskService(TaskRepository taskRepository) { this.taskRepository = taskRepository; }
```

This promotes loose coupling, high testability, and single responsibility principles.

Q2: What is the difference between JPA, Hibernate, and Spring Data JPA?

Answer:

- **JPA (Jakarta Persistence API):** A Java specification/standard defining interfaces and annotations (`@Entity`, `@Id`, `@Table`) for ORM.
- **Hibernate:** The concrete ORM implementation of the JPA specification. It generates SQL queries, manages database sessions, and handles schema DDL auto-generation.
- **Spring Data JPA:** A data access abstraction layer built on top of JPA/Hibernate. It provides standard repository interfaces (`JpaRepository`) so developers can execute CRUD operations without writing repetitive boilerplate SQL.

Q3: How does Spring Boot Auto-Configuration work in `TaskPulseApplication.java`?

Answer: The `@SpringBootApplication` annotation is a convenience annotation combining three annotations:

1. `@SpringBootConfiguration`: Designates the class as a Spring configuration bean.
2. `@EnableAutoConfiguration`: Scans classpath JARs (e.g. H2, Spring Web, Hibernate) and auto-configures Tomcat, DispatcherServlet, and DataSource based on default rules.
3. `@ComponentScan`: Scans current and child packages for Spring stereotypes (`@RestController`, `@Service`, `@Repository`, `@Component`).

Q4: Explain how CORS is configured in `WebConfig.java` and why it is necessary.

Answer: Cross-Origin Resource Sharing (CORS) is a browser security mechanism that restricts a webpage on one origin (`localhost:5173`) from making HTTP requests to a server on a different origin (`localhost:8080`).

We configured CORS in `WebConfig.java` by implementing `WebMvcConfigurer` and overriding `addCorsMappings()`:
`registry.addMapping("/api/**").allowedOriginPatterns("*").allowedMethods("GET", "POST", "PUT", "PATCH", "DELETE");`

This allows the React Vite frontend to perform REST operations without encountering browser cross-origin blocking.

Q5: What is the purpose of `@Enumerated(EnumType.STRING)` on `TaskStatus` and `TaskPriority`?

Answer: By default, JPA maps Java enums to database columns using `EnumType.ORDINAL` (storing numeric indices like 0, 1, 2). If an enum value is added or reordered, numeric mapping corrupts database data.

Using `@Enumerated(EnumType.STRING)` forces JPA to store explicit string literals (e.g. `'IN_PROGRESS'`, `'URGENT'`) in database columns, making the database robust, readable, and safe from enum reordering.

Q6: How does the `@PreUpdate` entity lifecycle hook work in `Task.java`?

Answer: JPA entity lifecycle annotations hook into Hibernate persistence events. In `Task.java`, the method: `@PreUpdate public void preUpdate() { this.updatedAt = LocalDateTime.now(); }` is automatically invoked by JPA right before an update SQL query is sent to the database, ensuring `updatedAt` is always synchronized accurately without manual boilerplate code.

Q7: How does state management and component communication work in our React frontend?

Answer: Our React application uses top-down state management in `App.jsx`:

- **Lifting State Up:** `App.jsx` holds central state (`tasks`, `stats`, `searchQuery`, `statusFilter`, `activeView`, `toasts`).
- **Props Passing:** `App.jsx` passes state down to child components (`Sidebar`, `Navbar`, `MetricsOverview`, `KanbanBoard`, `TaskTable`, `TaskModal`).
- **Callback Props:** Child components trigger parent state updates via callback functions passed as props (e.g. `onStatusChange`, `onSaveTask`, `onDeleteTask`).

Q8: Why did we use `useCallback` for `loadData()` in `App.jsx`?

Answer: In React, functions defined inside components are recreated on every render. Passing an inline function to `useEffect` dependency arrays causes infinite re-render loops.

Wrapping `loadData` in `useCallback(async () => { ... }, [statusFilter, priorityFilter, searchQuery])` memoizes the function instance so it is only recreated when one of its actual filter dependencies changes.

Q9: How does the Vite Proxy configuration in `vite.config.js` work?

Answer: During local development, the React frontend runs on port 5173 and the Spring Boot backend runs on port 8080. In `vite.config.js`, we configured:

```
proxy: { '/api': { target: 'http://localhost:8080', changeOrigin: true } }
```

Any request starting with `/api` is transparently proxied by the Vite dev server to Spring Boot, preventing CORS issues during local development.

Q10: How is Glassmorphism implemented in `index.css`?

Answer: Glassmorphism is achieved using CSS visual effects:

1. `background: rgba(15, 21, 39, 0.7);` — Semi-transparent dark background.
2. `backdrop-filter: blur(20px);` — Blurs elements underneath the panel.
3. `border: 1px solid rgba(255, 255, 255, 0.08);` — Subtle translucent border.
4. `box-shadow: 0 15px 35px -10px rgba(0, 0, 0, 0.6);` — Soft depth shadow.

Q11: How does Single-JAR Executable Packaging work for Spring Boot + React?

Answer: We compile the React frontend into static HTML/JS/CSS assets via `npm run build` (outputting to `frontend/dist`). We then copy these static assets into `backend/src/main/resources/static`.

When Maven runs `mvn clean package`, Spring Boot embeds these static files inside the executable JAR under `BOOT-INF/classes/static`. Spring Boot's `WelcomePageHandlerMapping` automatically serves `index.html` as the default root page on port 8080, packaging both frontend and backend into a single executable binary.

Q12: Explain the multi-stage build process in our production `Dockerfile`.

Answer: Multi-stage Docker builds reduce container size and security vulnerability surfaces:

- **Stage 1 (Node 20 Alpine):** Copies `package.json`, installs dependencies, and runs `npm run build` to generate compiled static bundle.
- **Stage 2 (Maven 3.9 + Java 21):** Copies Java source code and Stage 1 static bundle into Spring Boot resources, running `mvn clean package` to build the final runnable JAR.
- **Stage 3 (Eclipse Temurin JRE 21 Alpine):** Minimal runtime container copying only the output `app.jar` from Stage 2. Discards Node, npm, Maven, and source code, resulting in a lightweight, production-ready image.

■ Key Elevator Pitch for Your Interview:

"VortiQ Studio is an enterprise full-stack task management platform built with Spring Boot 3.3 REST APIs and React 18. I implemented a layered architecture with Spring Data JPA and H2 database, designed a modern glassmorphic UI with drag-and-drop Kanban workflow views, and containerized the application using multi-stage Docker builds and standalone single-JAR executable packaging."